Table des matières

Exercices corrigés 03 — Monter le projet dbt et le STAGING 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 3
Mises en situation 4
Corrigé 5

Exercices corrigés 03 — Monter le projet dbt et le STAGING

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur ce que vous venez de faire au labo ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 03 et du Labo 03.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Un modèle. Dites ce que contient un fichier de modèle dbt, et ce qu'il ne contient jamais.
2. Ce que dbt ajoute. Nommez les quatre choses que dbt apporte autour d'un `SELECT`, et dites laquelle relève du moteur de base de données.
3. Le nom de l'objet. D'où vient le nom de la table ou de la vue que dbt crée ?
4. Les deux fichiers. Distinguez `dbt_project.yml` et `profiles.yml` : ce que chacun déclare, et lequel se versionne.
5. Pourquoi ailleurs. Sous DuckDB, `profiles.yml` ne porte qu'un chemin de fichier. Donnez la raison pour laquelle il reste quand même hors du dépôt.
6. La cascade. Expliquez comment les réglages du bloc `models:` s'appliquent, et ce que cela évite d'écrire.
7. Une source. Définissez ce qu'est une source pour dbt, et l'engagement qu'on prend en la déclarant.
8. Quatre gains. Écrire `from raw.erp_kna1` fonctionne. Nommez ce que `source()` apporte de plus, et celui des gains qui se mesure le plus vite.

© 2026 iAlexMG. Tous droits réservés.

9. Le schéma qui s'ajoute. Un profil vise `staging`, un dossier déclare `+schema: marts`. Où dbt écrit-il, et pourquoi ce comportement existe-t-il ?
10. La macro. Dites ce que fait `generate_schema_name` dans la version de ce cours, en deux branches.
11. Les quatre gestes. Nommez les quatre gestes du `STAGING`, avec un exemple de chacun chez Boréal.
12. Le geste refusé. Quel geste le `STAGING` refuse-t-il, et quelle est la raison de fond ?
13. Le geste oublié. Des quatre gestes, lequel oublie-t-on, et que se passe-t-il si on l'oublie ?
14. L'état courant. Expliquez comment on tire l'état courant d'une table livrée par lots, en nommant la fonction employée.
15. Ce que l'état courant répare. Chez Boréal, un contrôle des commandes orphelines rend 0. Dites pourquoi ce zéro est le produit d'un choix.
16. Quatre matérialisations. Nommez-les, avec ce que dbt écrit dans chaque cas.
17. Le staging en vues. Donnez les trois raisons qui justifient de matérialiser le staging en vues.
18. `ref()`. Dites ce que cette fonction remplace, et les trois choses que dbt en tire.
19. Nommer en dur. Un modèle qui écrit `from staging.stg_commandes` au lieu de `ref()` fonctionne. Dites ce qui casse.
20. Trois mots proches. Distinguez, sur Snowflake, une table transient, le schéma `STAGING` d'un projet dbt, et un stage.
21. [labo] Le fichier vide. Que se passe-t-il si `profiles.yml` existe mais est vide, et en quoi est-ce pire qu'un fichier absent ?
1. [labo] Les deux comptes. `stg_commandes` rend 25 483 lignes là où sa source en porte 26 538. Expliquez l'écart.

Questions pratiques

P1. [labo] Une vue de plus. Écrivez `stg_articles.sql`, la vue de staging des articles, à partir de `raw.erp_mara`. Elle doit tirer l'état courant, décoder la gamme et le type d'article d'après le dictionnaire, et traiter l'année-modèle vide comme une absence.

P2. Vérifier la vue. Écrivez la requête qui compte les lignes de la source, celles de la vue, et les gammes non décodées. Donnez les trois résultats attendus.

P3. Déclarer sa source. Donnez le YAML à ajouter dans `_sources.yml` pour que la question P1 soit possible, et dites ce qui se passe si vous l'oubliez.

P4. Voir le SQL compilé. Donnez la commande qui produit le SQL réellement envoyé à DuckDB pour un modèle, dites où le fichier atterrit, et ce que `source()` y est devenu.

P5. Lire l'aval. Donnez la commande qui liste `stg_commandes` et tout ce qui en dépend. Donnez le résultat attendu sur le projet du labo.

© 2026 iAlexMG. Tous droits réservés.

P6. Lire l'amont. Donnez la commande qui liste tout ce dont `ctl_commandes_sans_concessionnaire` dépend, sources comprises, et le résultat attendu.

P7. Changer une matérialisation. Donnez les deux façons de matérialiser `stg_articles` en table plutôt qu'en vue, et la requête qui vérifie que c'en est bien une.

P8. Le contrôle du décodage. Écrivez la requête qui, sur `stg_articles`, vérifie d'un coup que le décodage de la gamme et celui du type d'article ont fonctionné.

Diagnostics

E1. Le graphe invisible. Un collègue écrit ce modèle de contrôle :

```
select count(*) as commandes
from staging.stg_commandes
```

Il tourne, `dbt run` rend `PASS=1`, et le résultat est juste. Dites ce qui ne va pas, comment le constater en une commande, et les deux conséquences concrètes.

E2. Le staging naïf. Un collègue écrit sa vue de staging des commandes ainsi :

```
select
    vbeln as numero_commande,
    kunnr as code_concessionnaire,
    netwr::decimal(18, 2) as valeur_nette
from {{ source('raw', 'erp_vbak') }}
```

Elle tourne, et le total du chiffre d'affaires part en production. Dites ce qui manque, de combien le total est faux, et pourquoi personne ne le verra.

E3. Le schéma doublé. Un collègue monte son projet, lance `dbt run`, et trouve ses vues dans un schéma `staging_staging`. Dites la cause et la correction.

E4. La jointure commode. Un collègue propose d'ajouter le nom du concessionnaire à `stg_commandes`, par une jointure vers `stg_concessionnaires` : « ça évitera de la refaire partout ensuite ». Répondez-lui, en donnant le principe et ce qui casse concrètement plus tard.

E5. Le chemin qui bouge. Un collègue lance `dbt run` depuis son dossier de projet, tout va bien. Le lendemain, il le lance depuis le dossier parent avec `--project-dir`, et reçoit une erreur de fichier introuvable sur la base. Dites la cause et la correction.

Questions à développer

A. Ce que dbt ne fait pas. On présente souvent dbt comme l'outil qui « transforme les données ». Expliquez en quoi cette formule est trompeuse, ce que dbt fait réellement, et ce que cela implique quand un modèle est lent.

© 2026 iAlexMG. Tous droits réservés.

B. Le staging qui ne joint pas. La règle paraît arbitraire, et elle coûte du travail : sans jointure, chaque consommateur du staging doit refaire les siennes. Défendez la règle, puis donnez le cas limite où vous accepteriez de la contourner et à quelles conditions.

C. (avancé) Déclarer ou pas. Ce cours déclare trois sources sur les vingt-deux tables de la `RAW`, au motif qu'on ne déclare que ce qu'on lit. Discutez ce choix : ce qu'il protège, ce qu'il coûte au moment où le projet grandit, et ce que vous feriez dans une équipe de cinq personnes.

Mises en situation

M1. Fromagerie de Saint-Prime. La fromagerie reçoit chaque semaine, de son distributeur, un fichier des ventes en épicerie : point de vente, produit, semaine, quantité. Le distributeur renvoie le fichier complet des douze dernières semaines à chaque livraison, si bien que la même semaine arrive douze fois, parfois avec des chiffres révisés. On vous demande de monter le staging. Dites ce que vous écrivez, quelle colonne fait office de lot, et le contrôle que vous posez avant de livrer quoi que ce soit à l'aval.

M2. (avancé) Scierie de Notre-Dame-du-Lac. Vous reprenez un projet dbt existant, quarante modèles. Vous découvrez que la moitié nomme les tables sources en dur, sans `source()`, et qu'un tiers des modèles se réfèrent entre eux sans `ref()`. Le projet tourne pourtant tous les matins. Décrivez ce que vous corrigez en premier, comment vous vous y prenez sans tout casser, et ce que vous pouvez promettre à votre chef pendant la reprise.

M3. Le staging qui ralentit. Chez un client, le staging est en vues, comme ici, mais une de ces vues est lue par une trentaine de modèles en aval, et chaque lecture relit huit millions de lignes brutes. L'exécution quotidienne passe de dix minutes à une heure. Dites ce que vous changez, ce que cela coûte, et le critère que vous retiendrez désormais pour choisir une matérialisation.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

Comment lire ce corrigé. Il donne la réponse et le pourquoi, jamais une notion neuve : tout ce qu'il emploie a été enseigné dans la Théorie 03 ou pratiqué dans le Labo 03. Quand une réponse repose sur un geste du labo, le renvoi le nomme.

Questions courtes

1. Un modèle. Un `SELECT`, et rien d'autre. Pas de `create table`, pas d'`insert`, pas de point-virgule final : c'est dbt qui écrit le `create` autour, selon la matérialisation choisie.
2. Ce que dbt ajoute. Un fichier par modèle dans un dépôt Git, un graphe de dépendances déduit du code, des tests déclarés à côté du modèle, une documentation générée depuis le code. Aucune des quatre ne relève du moteur : dbt ne calcule rien, il compile et envoie.
3. Le nom de l'objet. Du nom du fichier. `stg_concessionnaires.sql` produit `stg_concessionnaires`.
4. Les deux fichiers. `dbt_project.yml` fait exister le projet et porte ses réglages, dont la cascade du bloc `models:` ; il se versionne. `profiles.yml` dit comment se connecter à la base ; il ne se versionne pas et vit hors du dépôt.
5. Pourquoi ailleurs. Parce que la règle doit tenir avant de compter. Sous DuckDB le fichier ne porte qu'un chemin, mais les annexes Snowflake et Databricks lui font porter un compte, un utilisateur et de quoi s'authentifier. Une règle qu'on applique seulement quand elle compte ne s'applique jamais : personne ne se souvient de la rétablir au bon moment.
6. La cascade. Les réglages descendent du projet vers le dossier, puis vers le fichier, et le plus précis l'emporte. Cela évite de répéter sur chaque modèle ce qui vaut pour tout un dossier : on dit une fois que `models/staging/` est en vues.
7. Une source. Une table que vous n'avez pas fabriquée et que vous vous engagez à ne pas modifier. Chez Boréal, les tables de la couche `RAW`.
8. Quatre gains. Le graphe remonte jusqu'à la source, le nom du schéma vit à un seul endroit, la documentation connaît les sources, et la fraîcheur devient vérifiable. Celui qui se mesure le plus vite est le deuxième : un projet qui écrit le schéma en dur dans quarante modèles ne se déplace pas.
9. Le schéma qui s'ajoute. Dans `staging_marts` : dbt concatène le schéma du profil et celui du dossier. Le comportement existe parce que, sur une plateforme partagée, il évite que deux développeurs écrivent dans le même schéma.
10. La macro. Deux branches. Si le modèle ne déclare aucun schéma, prendre celui du profil ; s'il en déclare un, le prendre tel quel, sans concaténer. *Labo : `Labo_03_AE.md`, étape 3.*
11. Les quatre gestes. Renommer (`KUNNR` devient `code_concessionnaire`), typer (la date SAP devient une date, le montant devient un décimal), décoder (`01` devient Or), et tirer l'état courant (des trois livraisons d'une commande, garder la plus récente).
12. Le geste refusé. Joindre une autre source. La raison de fond est qu'une jointure est une décision de modélisation, et que la modélisation a ses couches : un `STAGING` qui joint devient un endroit où le métier se fait, en dehors de l'endroit prévu pour lui.

© 2026 iAlexMG. Tous droits réservés.

13. Le geste oublié. Tirer l'état courant. Si on l'oublie, les doublons de livraison de la `RAW` traversent toute la chaîne, et chaque total est faux de quelques pour cent sans que rien ne le signale. Mesuré au diagnostic E2 : 5,51 pour cent sur le chiffre d'affaires de Boréal.

14. L'état courant. Par une fonction de fenêtre : `row_number()` partitionne par la clé, trie par `_lot` en ordre décroissant, et le `where rang = 1` ne garde que la plus récente. *Labo : `Labo_03_AE.md`, étape 7.*

15. Ce que l'état courant répare. Parce qu'on a choisi de regarder le référentiel d'aujourd'hui plutôt que celui de l'époque. Un concessionnaire de Boréal a commandé avant que sa fiche n'arrive : à l'époque, sa commande était orpheline. Sur l'état courant, la fiche existe et le problème a disparu. *Labo : `Labo_03_AE.md`, étape 8.*

16. Quatre matérialisations. `view`, dbt écrit une vue ; `table`, il écrit une table ; `ephemeral`, il n'écrit rien et injecte le SQL dans les modèles qui l'appellent ; `incremental`, il écrit une table qu'il complète par tranches.

17. Le staging en vues. Une vue ne coûte rien à construire, elle est toujours à jour, et le staging ne fait que renommer et typer, donc son coût de lecture est négligeable.

18. `ref()`. Elle remplace le nom en dur d'un autre modèle. dbt en tire l'ordre d'exécution, ce qui doit être reconstruit quand un modèle change, et la documentation du lignage.

19. Nommer en dur. Le graphe. dbt ne voit pas le lien, donc il peut exécuter les deux modèles dans le mauvais ordre, et la documentation ment sur le lignage. Mesuré au diagnostic E1.

20. Trois mots proches. Une table transient, sur Snowflake, est une table sans protection de récupération après suppression : moins chère à stocker, perdue définitivement si on l'efface. Le schéma `STAGING` d'un projet dbt est une couche de modélisation, celle qui rend les sources lisibles. Un stage Snowflake est un emplacement de fichiers avant chargement. Aucun rapport entre les trois, malgré la ressemblance des mots.

21. Le fichier vide. dbt 1.12.3 s'arrête sur une trace Python et le message `cannot access local variable 'summary_message'`, qui ne dit rien. C'est pire qu'un fichier absent, parce qu'un fichier absent donne un message clair, qui nomme le fichier cherché et l'endroit où dbt l'a cherché. *Labo : `Labo_03_AE.md`, étape 1.*

22. Les deux comptes. La `RAW` garde chaque livraison : 26 538 lignes pour 25 483 commandes, soit 1 055 lignes en trop. Deux causes : 629 viennent du lot de mars 2023 livré deux fois, qui touche 609 commandes dont vingt apparaissent ainsi trois fois, et 426 viennent de commandes revenues dans un extrait ultérieur après modification. L'état courant n'en garde qu'une par commande, et aucune commande n'est perdue. *Labo : `Labo_03_AE.md`, étape 7.*

Questions pratiques

P1. Une vue de plus. Dans `models/staging/stg_articles.sql` :

```
with courant as (
```


© 2026 iAlexMG. Tous droits réservés.

```
    select
        *,
        row_number() over (
            partition by matnr
            order by _lot desc
        ) as rang

    from {{ source('raw', 'erp_mara') }}

)

select
    matnr as code_article,
    case matkl
        when 'MN' then 'Motoneige'
        when 'MM' then 'Motomarine'
        when 'HR' then 'Hors-route'
        when 'PAC' then 'Pièces et accessoires'
    end as gamme,
    case mtart
        when 'FERT' then 'Véhicule'
        when 'HAWA' then 'Pièce'
    end as type_article,
    nullif(zzannee_mod, '') as annee_modele,
    meins as unite_de_base,
    _lot as lot_source

from courant
where rang = 1
```

Les quatre gestes, et rien d'autre. Le `nullif` sur l'année-modèle est le point à ne pas manquer : elle est vide pour une pièce, et une chaîne vide n'est pas une absence.

P2. Vérifier la vue.

```
select
    (select count(*) from raw.erp_mara) as lignes_dans_la_raw,
    (select count(*) from staging.stg_articles) as articles_dans_le_staging,
    (select count(*) from staging.stg_articles where gamme is null) as gammes_vides;
```

Attendu : 2 459 lignes dans la `RAW`, 68 articles dans le staging, 0 gamme vide. Le rapport est brutal, trente-six livraisons par article en moyenne, parce que le référentiel complet est livré à chaque extrait mensuel.

© 2026 iAlexMG. Tous droits réservés.

P3. Déclarer sa source. Dans `_sources.yml`, sous `tables:` :

```
      - name: erp_mara
        description: "Les articles."
```

Sans cette déclaration, le modèle ne compile pas : `Compilation Error ... depends on a source named 'raw.erp_mara' which was not found.` dbt refuse de deviner, et c'est le comportement qu'on veut.

P4. Voir le SQL compilé.

```
dbt compile --select stg_articles
```

Le fichier atterrit dans `target/compiled/boreal_ae/models/staging/stg_articles.sql`. En l'ouvrant, on voit que `{{ source('raw', 'erp_mara') }}` est devenu `"boreal"."raw"."erp_mara"` : le nom complet, avec la base, le schéma et la table. C'est la commande à lancer quand on ne comprend pas ce que dbt exécute vraiment.

P5. Lire l'aval.

```
dbt ls --select stg_commandes+
```

Attendu, deux lignes : `boreal_ae.staging.stg_commandes` et `boreal_ae.staging.ctl_commandes_sans_concessionnaire`. Le `+` après un nom veut dire « et tout ce qui en dépend ». C'est la commande qui répond à « qu'est-ce que je casse si je change ce modèle ».

P6. Lire l'amont.

```
dbt ls --select +ctl_commandes_sans_concessionnaire
```

Attendu, cinq lignes : le modèle lui-même, les deux vues de staging dont il dépend, et les deux sources dont celles-ci dépendent, `source:boreal_ae.raw.erp_kna1` et `source:boreal_ae.raw.erp_vbak`. Le `+` devant le nom veut dire « et tout ce dont il dépend ». C'est la commande qui répond à « d'où vient ce chiffre ».

P7. Changer une matérialisation. Deux façons, et elles ne se valent pas.

Dans le `dbt_project.yml`, si le changement vaut pour tout un dossier :

```
models:
  boreal_ae:
    staging:
      +materialized: table
```

Ou dans le modèle lui-même, en première ligne, si le changement ne vaut que pour lui :

```
{{ config(materialized='table') }}
```

La seconde l'emporte sur la première, par la cascade. Pour vérifier :

© 2026 iAlexMG. Tous droits réservés.

```sql
select table_name, table_type
from information_schema.tables
where table_schema = 'staging'
order by 1;
```

La colonne passe de `VIEW` à `BASE TABLE` pour le modèle visé.

P8. Le contrôle du décodage.

```sql
select
    count(*) as articles,
    count(*) filter (where gamme is null) as gammes_non_decodees,
    count(*) filter (where type_article is null) as types_non_decodes,
    count(*) filter (where annee_modele is null) as sans_annee_modele
from staging.stg_articles;
```

Attendu : 68 articles, 0 gamme non décodée, 0 type non décodé, 23 sans année-modèle. Les deux zéros sont des contrôles ; le 23 n'en est pas un, c'est le compte des pièces et accessoires, qui n'ont pas d'année-modèle. Savoir lequel des trois est un contrôle et lequel est un fait métier est tout le métier.

Diagnostics

E1. Le graphe invisible. Le modèle nomme `staging.stg_commandes` en dur, sans `ref()`. dbt ne voit donc aucune dépendance, ce qui se constate en une commande :

```
dbt ls --select +ctl_en_dur
```

Elle rend une seule ligne, le modèle lui-même, là où elle devrait en rendre trois.

Deux conséquences. L'ordre d'exécution n'est plus garanti : dbt peut construire ce contrôle avant la vue dont il dépend, et il échouera ou lira un état périmé, selon les jours. Et la documentation ment sur le lignage : personne, en regardant le graphe, ne saura que ce contrôle dépend des commandes.

Le plus vicieux est que tout fonctionne aujourd'hui, parce que la vue existe déjà.

E2. Le staging naïf. Il manque l'état courant : le modèle lit les 26 538 lignes de la `RAW` au lieu des 25 483 commandes.

Mesuré sur Boréal : la somme des valeurs nettes passe de 430 148 012,16 $ à 453 849 040,59 $, soit 23 701 028,43 $ de trop, ou 5,51 pour cent.

Personne ne le verra, et c'est le cœur du problème. Le chiffre n'est ni absurde ni rond : un chiffre d'affaires de 453 millions pour un fabricant de véhicules récréatifs est parfaitement plausible, et il ne sera comparé à rien qui le contredise. L'erreur ne se découvre que le jour où quelqu'un recompte depuis l'ERP, souvent des mois plus tard.

© 2026 iAlexMG. Tous droits réservés.

E3. Le schéma doublé. La macro `generate_schema_name` manque, ou n'est pas dans le dossier déclaré par `macro-paths`. Sans elle, dbt concatène le schéma du profil, `staging`, et celui du dossier, `staging`. La correction est d'écrire la macro de l'étape 3 du labo, puis de relancer `dbt run` ; les vues du schéma doublé restent dans la base et se suppriment à la main.

E4. La jointure commode. Le principe d'abord : au staging, décoder une colonne oui, joindre une autre table non. Une vue de staging décrit une source, pas une relation entre deux sources.

Ce qui casse concrètement plus tard, et c'est ce qui convaincra : cette jointure fige la version courante du concessionnaire. Le jour où la dimension passe en type 2, au Cours 05, tout modèle qui consomme cette vue hérite du nom d'aujourd'hui, et non de celui de l'époque de la commande. La jointure aura essaimé dans dix modèles entre-temps, et il faudra la défaire partout. Le gain de confort d'aujourd'hui se paie en reprise dans six mois.

E5. Le chemin qui bouge. Le `path` de `profiles.yml` est relatif, et dbt le résout depuis le répertoire où la commande est lancée, pas depuis le projet. Lancé du dossier parent, il cherche la base un cran trop haut, d'où l'erreur `IO Error: Cannot open file ... Le chemin d'accès spécifié est introuvable.` La correction est d'écrire un chemin absolu, ce qui ne coûte rien puisque le fichier est propre au poste et ne se versionne pas.

Questions à développer

A. Ce que dbt ne fait pas. La formule est trompeuse parce qu'elle laisse croire que dbt est un moteur de traitement, alors qu'il n'exécute aucun calcul. Ce qu'il fait réellement : il lit vos fichiers, remplace le Jinja par du SQL, déduit un graphe des appels à `ref()` et `source()`, ordonne les modèles, puis envoie chaque requête à la base et lit la réponse. Tout le travail se fait dans DuckDB, ou dans Snowflake, ou dans Databricks.

Ce que cela implique quand un modèle est lent tient en une phrase : le problème est dans la base, et la solution aussi. Aucun réglage de dbt ne rendra une requête plus rapide, parce que dbt n'a rien à optimiser. Il faut regarder le plan d'exécution, la volumétrie, les types, l'ordre des jointures, exactement comme on le ferait sans dbt. La seule chose que dbt permet, et elle compte, c'est de changer la matérialisation : transformer une vue relue trente fois en table calculée une fois déplace le coût, sans rien changer au SQL.

Renvoi : les plans d'exécution et l'optimisation sont au Cours 10 de SQL.

B. Le staging qui ne joint pas. La défense tient en trois arguments. Une vue de staging qui joint n'a plus de grain clair : elle décrit une commande, ou un concessionnaire, ou les deux, et personne ne sait plus ce qu'une ligne représente. Elle fige des choix qui appartiennent à la modélisation, comme celui de la version d'une dimension. Et elle rend le staging non réutilisable : un consommateur qui n'a pas besoin du concessionnaire paie quand même la jointure.

Le cas limite que j'accepterais est celui d'une source livrée en plusieurs fichiers qui ne forment, ensemble, qu'une seule entité logique : un en-tête et son détail découpés par contrainte technique, qui n'ont aucun sens séparément. Réunir ces deux fichiers n'est pas une jointure de modélisation, c'est la reconstitution d'une source. La condition est de pouvoir l'écrire ainsi : si vous ne pouvez pas dire « ces deux fichiers sont une seule table que l'exportateur a coupée », c'est une jointure de modélisation, et elle appartient à l'`INTERMEDIATE`.

C. (avancé) Déclarer ou pas. Le choix de ne déclarer que ce qu'on lit protège d'une chose précise : une source déclarée est une promesse, et dbt la tient pour acquise. Elle apparaît dans la documentation, elle entre dans le périmètre des contrôles de fraîcheur, et elle donne à penser qu'un modèle en dépend. Déclarer

© 2026 iAlexMG. Tous droits réservés.

les vingt-deux tables alors qu'on en lit trois produit un lignage qui montre dix-neuf sources orphelines, et un lecteur ne sait plus distinguer ce qui sert de ce qui attend.

Ce que cela coûte quand le projet grandit : chaque nouveau modèle demande de vérifier si sa source est déjà déclarée, et le fichier de sources se remplit par petites touches, souvent dans le même commit que le modèle. C'est un frottement réel mais faible, et il a une vertu : il oblige à écrire une description au moment où l'on sait ce que la table contient.

Dans une équipe de cinq personnes, je garderais la règle et j'ajouterais une convention : un fichier de sources par système source plutôt qu'un seul fichier pour tout, ce qui réduit les conflits de fusion quand deux personnes déclarent une table le même jour. Et je poserais la description comme obligatoire, parce que c'est le seul moment où quelqu'un la rédigera.

Mises en situation

M1. Fromagerie de Saint-Prime. Le cas est exactement celui de Boréal, en plus systématique : chaque livraison renvoie douze semaines, donc chaque semaine arrive douze fois, avec des chiffres qui peuvent différer d'une livraison à l'autre.

Ce que j'écris est une vue de staging qui tire l'état courant, avec pour clé le triplet point de vente, produit et semaine, et pour ordre la date de livraison décroissante. La colonne qui fait office de lot est la date de réception du fichier, et si le distributeur ne la fournit pas, c'est le nom du fichier ou l'horodatage du chargement qui la porte : sans elle, la règle « la plus récente gagne » n'est pas applicable, et c'est la première chose à réclamer.

Le contrôle que je pose avant de livrer quoi que ce soit est celui du grain, comparer le nombre de lignes au nombre de triplets distincts, et un second qui compte les révisions : pour chaque semaine, combien de livraisons différentes l'ont portée, et de combien le chiffre a bougé entre la première et la dernière. Ce second contrôle n'est pas un test qui échoue, c'est une mesure à montrer au métier : si les révisions sont fortes, tout rapport hebdomadaire publié avant douze semaines est provisoire, et il faut le dire.

M2. (avancé) Scierie de Notre-Dame-du-Lac. Ce que je corrige en premier n'est ni l'un ni l'autre : c'est l'établissement de l'état des lieux. `dbt ls --select +<modèle>` sur les modèles de sortie dit immédiatement lesquels ont un graphe complet et lesquels sont orphelins ; la liste des modèles dont l'amont est vide est exactement la liste du travail à faire.

Ensuite, les `ref()` manquants avant les `source()` manquants. La raison est que les `ref()` manquants mettent l'exécution en danger tous les matins, par un ordre non garanti, tandis qu'un `source()` manquant ne fait que dégrader la documentation et empêcher les contrôles de fraîcheur. Le risque n'est pas du même ordre.

Comment m'y prendre sans tout casser : un modèle à la fois, en vérifiant après chaque changement que le SQL compilé est identique. C'est ce que `dbt compile` permet, et c'est la garantie qu'on ne change que le graphe, pas le résultat. Un remplacement de `staging.stg_x` par `{{ ref('stg_x') }}` doit produire exactement le même SQL compilé, aux guillemets près.

© 2026 iAlexMG. Tous droits réservés.

Ce que je peux promettre à mon chef pendant la reprise : que rien ne changera dans les chiffres, et que je peux le prouver modèle par modèle. Ce que je ne promets pas : un calendrier ferme, tant que je n'ai pas fini l'état des lieux, parce que quarante modèles dont on ignore le graphe peuvent cacher des dépendances circulaires qui ne se découvrent qu'en les déclarant.

M3. Le staging qui ralentit. Ce que je change : cette vue-là devient une table. Pas tout le staging, cette vue.

Ce que cela coûte, et il faut le dire honnêtement : la table doit être reconstruite à chaque exécution, donc le temps de construction s'ajoute une fois, et elle occupe de l'espace. Surtout, elle n'est plus toujours à jour : entre deux exécutions, elle porte l'état du dernier passage, alors qu'une vue reflétait la source à la seconde près. Sur un chargement quotidien, cela ne change rien ; sur une source qui bouge en continu, cela change tout, et il faut le savoir.

Le critère que je retiens désormais n'est pas la taille de la source, c'est le produit de deux choses : combien de fois la vue est lue dans une exécution, et combien coûte une lecture. Une vue lue une fois reste une vue, quelle que soit sa volumétrie. Une vue lue trente fois sur huit millions de lignes est une table, même si le SQL paraît trivial. C'est exactement la lecture que fait `dbt ls --select <modèle>+`, qui donne le premier des deux facteurs.

© 2026 iAlexMG. Tous droits réservés.